

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JNANA SANGAMA” Belagavi – 590018, Karnataka



A

PROJECT BASED ASSESSMENT

ON

“PredictCare: An ML-Enabled Decision Support System for Urgency-Aware Appointment Management”

Submitted in partial fulfillment for the award of the degree in

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

DN UNNATHI 1BH23CS014

KADAJJI MADHUSHREE 1BH23CS032

MADHUMITHA N 1BH23CS045

Under the Guidance of

Mrs. Shylaja D N

Assistant professor



Department of Computer Science and Engineering

BANGALORE TECHNOLOGICAL INSTITUTE

(An ISO 9001:2005 Certified Institute)

Kodathi Village, Varthoor Hobli, Banagalore East Tq, Bangalore Urban District

Bangalore – 560035, Karnataka

2025 – 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**CERTIFICATE**

This is to certify that the project-based assessment ON “ **PREDICTCARE: AN ML-ENABLED DECISION SUPPORT SYSTEM FOR URGENCY-AWARE APPOINTMENT MANAGEMENT**” carried out by the **DN UNNATHI[1BH23CS014], KADAJJIMADHUSHREE[1BH23CS032], MADHUMITHA N [1BH23CS045]** Bonafide student at **Bangalore Technological Institute**, Bengaluru, in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering, **Visvesvaraya Technological University, Belagavi**, during the academic year **2025-2026**. Thus, it is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The project-based assessment on cryptography report has been approved, as it satisfies the academic requirements with respect to the project-based assessment report prescribed for the said degree.

Details of the Batch	Context & clarity (12)	Technical Depth & Architecture (13)	Project Implementation & Execution (12)	Presentation & Viva (13)	Report (25)	Total (75)	Final (25)
Name: DN UNNATHI							
USN : 1BH23CS014							
Name: KADAJJI MADHUSHREE							
USN : 1BH23CS032							
Name: MADHUMITHA N							
USN:1BH23CS045							

Mrs. Shylaja D N

Assistant professor

Dept. of CSE

Dr. Jerline Sheeba Anni

HOD

Dept. of CSE

ABSTRACT

PredictCare: An ML-Enabled Decision Support System for Urgency-Aware Appointment Management is a web-based intelligent healthcare application designed to improve patient appointment scheduling and clinical prioritization by integrating machine learning with a Flask-based web platform. The system predicts patient urgency level and estimated waiting time using Random Forest Classifier and Random Forest Regressor algorithms trained on a structured healthcare dataset containing attributes such as age, symptom, and department. The project follows a methodology involving data generation/preparation, preprocessing, label encoding, model training, evaluation, and deployment into a real-time web application. The frontend is developed using HTML, CSS, and Bootstrap, while the backend is implemented using Python Flask, with SQLite used for database management and SMTP-based email services used for appointment notifications. The system includes features such as patient data entry, urgency prediction, waiting time estimation, doctor and hospital recommendation, appointment booking, slot availability management, and admin appointment viewing. By combining predictive analytics with healthcare workflow automation, PredictCare aims to support faster decision-making, improve appointment efficiency, and enhance the overall patient management process.

INDEX

TITLE	PAGE NO.
ABSTRACT	i
CONTENTS	ii
LIST OF FIGURES	iv
LIST OF TABLES	v

CONTENTS

SL.No	CHAPTERS	Page No.
1	INTRODUCTION	3
	1.1 Introduction	3
	1.2 Background	3
	1.3 Problem Statement	4
	1.4 Objectives	5
	1.5 Motivation	5
2	LITERATURE SURVEY	6
	2.1 Literature Survey Review	6
3	SYSTEM ANALYSIS	8
	3.1 Existing System	8
	3.2 Advantages of Existing System	8
	3.3 Disadvantages of Existing System	9
	3.4 Proposed System	9

SL.No	CHAPTERS	Page No.
	3.5 Feasibility Study	10
4	OBJECTIVES AND METHODOLOGY	11
	4.1 Objectives	11
	4.2 Methodology	11
5	REQUIREMENT ANALYSIS	15
	5.1 Hardware Requirements	15
	5.2 Software Requirements	15
6	IMPLEMENTATION	17
	6.1 System Architecture	17
	6.2 Modules	18
	6.3 Algorithm Workflow	19
7	RESULTS AND DISCUSSION	21
	7.1 Overview	21
	7.2 Results	22
	7.3 Discussion and Analysis	25
8	CONCLUSION	26
	8.1 Conclusion	26
	8.2 Future Enhancements	26
	REFERENCES	30
	APPENDIX A: SCREENSHOT	31
	APPENDIX: SOURCE CODE	33

LIST OF FIGURES

Figure No.	Title	Page No.
6.2	Workflow Representation	20
7.1	Login Page	22
7.2	Appointment Confirmation & Urgency Prediction	23
7.3	PredictCare Appointment Confirmation	23
7.4	PredictCare Admin Dashboard	24
7.5	PredictCare Admin Dashboard	24
A.1	Synthetic Healthcare Data Generation Module	28
A.2	Machine Learning Model Training Module	29
A.3	Urgency Prediction Code	30
A.4	Available Slots Code	31
A.5	Booking Validation Code	32

LIST OF TABLES

Table No.	Title	Page No.
1	Literature Survey	6

Chapter 1

INTRODUCTION

1.1 Introduction

In the modern healthcare environment, efficient patient management has become a critical factor in ensuring high-quality service delivery and improved clinical outcomes. With the rapid increase in population and healthcare demands, hospitals and medical centers are facing significant challenges in managing patient flow effectively. Issues such as overcrowding, prolonged waiting times, and inefficient allocation of medical resources have become increasingly common.

Traditional healthcare systems often rely on manual processes and fixed scheduling methods, which lack adaptability and intelligence. As a result, patients with severe medical conditions may experience delays in receiving timely care, leading to compromised treatment outcomes.

To address these challenges, PredictCare is designed as an intelligent, machine learning-based decision support system that enhances appointment scheduling and prioritization. The system leverages predictive analytics to assess patient urgency based on key parameters such as age, symptoms, and department.

By predicting the severity of a patient's condition and estimating waiting time, PredictCare ensures that high-priority cases are identified early and attended to promptly. This not only improves patient care but also optimizes hospital resource utilization, ultimately contributing to a more efficient healthcare system.

1.2 Background

Traditional appointment scheduling systems in healthcare institutions typically operate on simple, rule-based approaches such as:

- First-Come-First-Serve (FCFS)
- Manual scheduling by administrative staff

While these methods are easy to implement, they fail to address the dynamic nature of healthcare requirements. Specifically, they do not consider the urgency or severity of a patient's condition, which is a crucial factor in medical decision-making.

- Inability to prioritize critical patients
- Inefficient utilization of doctor availability
- Increased patient waiting time

- Lack of automation and intelligence

With the advancement of technologies such as Machine Learning and Artificial Intelligence, healthcare systems are now capable of analyzing large volumes of patient data to derive meaningful insights. These technologies enable predictive modeling, which can be used to forecast patient conditions and optimize decision-making processes.

PredictCare builds upon these advancements by integrating machine learning algorithms into the appointment management system. It utilizes data-driven techniques to predict urgency levels and waiting times, thereby transforming traditional scheduling into an intelligent and automated process.

1.3 Problem Statement

Despite the availability of modern healthcare facilities, existing appointment management systems suffer from several critical limitations that impact both patients and healthcare providers.

The major problems identified in current systems include:

- Lack of urgency-based patient prioritization
- Inefficient and rigid appointment scheduling mechanisms
- Increased waiting time for patients, including critical cases
- Overburdening of medical staff due to poor workload distribution
- Limited use of data-driven decision-making
- Reduced overall patient satisfaction

These challenges highlight the need for a smart and adaptive system that can overcome the

Therefore, there is a requirement for an intelligent healthcare system that can:
inefficiencies of traditional approaches.

- Predict patient urgency levels using advanced algorithms
- Estimate waiting time accurately
- Automate the scheduling process
- Assist healthcare professionals in decision-making
- Improve overall system efficiency and patient experience

PredictCare aims to address these issues by introducing a machine learning-based framework for urgency-aware appointment management.

1.4 Objectives

The primary objective of the PredictCare system is to develop an intelligent healthcare solution inefficiencies of traditional approaches.

The specific objectives of the project are:

- To design and implement a machine learning-based model for predicting patient urgency levels
- To estimate patient waiting time using regression techniques
- To develop a web-based application for automated appointment scheduling
- To provide personalized doctor and hospital recommendations based on patient data
- To integrate predictive models with a real-time healthcare management system
- To improve decision-making efficiency for healthcare providers
- To reduce patient waiting time and improve service quality
- To enhance patient satisfaction through better prioritization and management

1.5 Motivation

The motivation behind developing PredictCare arises from the increasing need for intelligent healthcare systems that can handle complex patient management scenarios efficiently.

Several factors contribute to the motivation of this project:

- Rapid growth in patient population and healthcare demand
- Need for faster and more accurate clinical decision-making
- Limitations of traditional manual scheduling systems
- Increasing workload on hospital staff and administrative personnel
- Growing importance of data-driven solutions in healthcare
- Demand for improved patient experience and satisfaction

Additionally, the integration of Machine Learning and Artificial Intelligence in healthcare presents a significant opportunity to revolutionize traditional systems. By automating processes and enabling predictive analysis, these technologies can significantly enhance efficiency and accuracy

Chapter 3

SYSTEM ANALYSIS

3.1 Existing System

The existing healthcare appointment management systems primarily rely on traditional and manual approaches for scheduling patient visits. These systems typically operate using simple methods such as manual booking or fixed scheduling rules, without incorporating any form of intelligence or predictive capability.

In most hospitals and clinics, appointment scheduling is handled through:

- Manual appointment booking by administrative staff
- First-Come-First-Serve (FCFS) scheduling approach
- Fixed time-slot allocation without considering patient condition

Such systems do not utilize patient data effectively and fail to analyze critical factors such as the severity of symptoms or urgency of treatment. As a result, all patients are treated equally in the scheduling process, regardless of their medical condition.

Moreover, the absence of predictive analytics in existing systems leads to inefficient handling of patient flow, increased waiting times, and potential delays in treating critical cases. The lack of integration with intelligent technologies further limits the system's ability to adapt to real-time scenarios.

3.2 Advantages of Existing System

Despite their limitations, traditional systems offer certain basic advantages that make them widely used:

- Easy to Implement.
- Minimal Infrastructure Requirements
- Low Technical Complexity
- Widely-Adopted

3.3 Disadvantages of the Existing System

The existing hospital appointment and patient management systems are primarily based on manual scheduling and basic rule-based processes.

1. Lack of Intelligent Decision-Making

The current system does not utilize artificial intelligence or machine learning techniques to analyze patient data

2. High Waiting Time

Since appointments are scheduled manually and there is no automated prioritization mechanism, patients often experience long waiting times.

3. Poor Resource Utilization

The existing system does not optimize the allocation of doctors, nurses, and medical resources.

4. No Automation

Most processes, including appointment booking, patient categorization, and scheduling, are handled manually.

5. Inability to Handle Critical Cases Efficiently

The system lacks a mechanism to automatically identify and prioritize patients with serious or life-threatening conditions.

6. Lack of Real-Time Adaptability

The current system cannot dynamically adjust to changes such as sudden increases in patient inflow, doctor unavailability, or emergency situations.

7. Limited Predictive Capability

Existing systems do not forecast patient no-shows, consultation duration, or department demand.

8. Reduced Patient Satisfaction

Long waiting times, scheduling inefficiencies, and delayed treatment contribute to poor patient experience

3.4 Proposed System

To overcome the limitations of the existing system, the proposed system, PredictCare, introduces an intelligent, automated, and data-driven approach to healthcare appointment management.

The proposed system integrates Machine Learning techniques with a web-based platform to provide real-time decision support. It utilizes predictive models to assess patient urgency and estimate waiting time, enabling efficient scheduling and prioritization.

Key Features of Proposed System include:

Urgency Prediction:

Classifies patients into urgency levels (low, medium, high) using predictive models.

Waiting Time Estimation:

Provides an estimated waiting time for each patient based on current system load.

Appointment Booking:

Allows users to book appointments seamlessly through the web interface.

Doctor and Department Recommendation:

Suggests suitable doctors and departments based on patient symptoms.

Email Notification System:

Sends appointment confirmation and updates via email.

Admin Dashboard:

Enables administrators to view and manage all appointments efficiently.

3.5 Feasibility Study

The feasibility study evaluates whether the proposed system is practical, implementable, and beneficial.

i. Technical Feasibility

The system is technically feasible as it uses widely available technologies like Python and Flask. It integrates machine learning libraries such as Scikit-learn, Pandas, and NumPy for prediction tasks. SQLite is used for lightweight database management. The frontend and backend components are easily integrated. Overall, the system is reliable, simple, and efficient to implement.

ii. Operational Feasibility

The system is user-friendly and easy for both patients and healthcare staff to operate. It has a simple and intuitive interface requiring minimal training. It reduces the workload on administrative staff. The system improves efficiency in managing appointments. It can be smoothly integrated into existing workflows without disruption.

iii. Economic Feasibility

The system is cost-effective as it uses open-source tools and libraries. It requires minimal hardware resources for operation. Development and maintenance costs are low. It helps reduce operational expenses by automating tasks. Overall, it provides a high return on investment by improving efficiency.

Chapter 4

OBJECTIVES AND METHODOLOGY

4.1 Objectives

The primary objective of the PredictCare system is to design and develop an intelligent healthcare solution that enhances patient appointment management through predictive analytics and automation.

The specific objectives of the system are as follows:

Develop a machine learning model that classifies patients into different urgency levels (low, medium, high) based on their input data such as age, symptoms, and department.

Missing or incomplete data is handled by either removing records or filling them using appropriate techniques such as mean, median, or mode.

Implement a regression-based model to calculate the expected waiting time for patients, helping them plan their visit more efficiently.

Missing or incomplete data is handled by either removing records or filling them using appropriate techniques such as mean, median, or mode.

Replace manual scheduling with an automated system that dynamically allocates appointment slots based on urgency and availability.

Enhance the overall efficiency of healthcare services by reducing waiting time, improving prioritization, and supporting faster clinical decision-making.

Ensure effective use of doctors, departments, and time slots by intelligently distributing patient load.

Assist hospital staff in making quick and informed decisions using predictive insights.

4.2 Methodology

The development of the PredictCare system follows a structured and systematic methodology that involves multiple stages, from data collection to deployment. Each stage plays a crucial role in ensuring the accuracy and effectiveness of the system.

Step-1:Data Collection

Data collection is the initial and most important step in building a machine learning model. The performance of the system largely depends on the quality and relevance of the dataset used.

The dataset used in this project includes the following attributes:

Age: Represents the age of the patient, which is an important factor in determining health risk levels.

Symptoms: Describes the symptoms experienced by the patient. This is a critical feature for predicting

urgency.

Department: Indicates the medical department required (e.g., cardiology, orthopedics, general medicine).

The dataset may be either:

Real-world healthcare data (if available), or

Simulated data generated for training purposes

The collected data is stored in a structured format such as CSV or database tables for further processing.

Step-2:Data Preprocessing

Raw data cannot be directly used for machine learning. It must be cleaned and transformed into a suitable format. This stage ensures that the dataset is consistent, complete, and ready for analysis.

The preprocessing steps include:

Handling Missing or incomplete data Values:

Missing or incomplete data is handled by either removing records or filling them using appropriate techniques such as mean, median, or mode

Encoding Categorical Data:

Since machine learning models require numerical input, categorical variables such as symptoms and departments are converted into numerical form using techniques like label encoding.

Feature

Relevant features are selected to improve model performance and reduce complexity. Irrelevant or redundant data is removed.

Data Normalization (if-required):

Ensures that all features are on a similar scale, improving model efficiency.

This step improves the quality of input data and enhances the accuracy of predictions.

Step-3:Model Training

In this phase, machine learning models are trained using the preprocessed dataset to learn patterns and relationships between input features and target outputs.

The system uses models based on Random Forest algorithm due to its high accuracy and robustness.

i. Random Forest Classifier

Used to predict the urgency level of patients

- Classifies patients into categories such as low, medium, and high urgency
- Works by creating multiple decision trees and combining their outputs

ii. Random Forest Regressor

- Used to predict the waiting time

- Provides continuous numerical output (time estimation)
- Improves prediction accuracy by averaging multiple decision trees

These models are trained using training data and validated using testing data to ensure proper learning.

Step-4: Model Evaluation

After training, the performance of the models is evaluated using standard evaluation metrics.

For Classification Model:

Accuracy: Measures how correctly the model predicts urgency levels.

For Regression Model:

Mean Squared Error (MSE):

Calculates the average squared difference between predicted and actual values.

Root Mean Squared Error (RMSE):

Provides error measurement in the same unit as the output variable, making it easier to interpret.

Step-5: Deployment

Once the model is trained and evaluated, it is integrated into a web-based application for real-time use.

- The backend is implemented using the Flask framework
- Machine learning models are loaded into the Flask application
- User inputs are processed and passed to the models for prediction

Step-6: Web Interface:

- Developed using HTML, CSS, and Bootstrap
- Provides forms for entering patient details
- Displays predicted urgency level and waiting time

Step-7: System Workflow:

- User enters patient details
- Data is processed and encoded
- Model predicts urgency and waiting time
- Results are displayed on the interface
- Appointment is scheduled automatically

Deployment ensures that the system is accessible, interactive, and usable in real-world scenarios.

Chapter 5

REQUIREMENT ANALYSIS

5.1 Hardware Requirements

The PredictCare system is designed to run efficiently on standard computing devices without requiring high-end hardware. The following are the minimum hardware requirements:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB
- Input/Output Devices:

These requirements make the system accessible and deployable even in small-scale healthcare setups.

5.2 Software Requirements

The system is developed using widely available and open-source software technologies:

- Programming Language: Python

Used for backend development and machine learning model implementation.

- Web Framework: Flask

Provides a lightweight framework for building the web application.

- Frontend Technologies:
 - HTML (structure)
 - CSS (styling)
 - Bootstrap (responsive design)

- Database:

Used for storing patient details and appointment records.

- Machine Learning Libraries:
 - Scikit-learn (model building)
 - Pandas (data manipulation)
 - NumPy (numerical computations)

Chapter 6

IMPLEMENTATION

6.1 System Architecture

The PredictCare system follows a two-layer architecture, which separates the user interface from the backend processing logic. This modular design improves scalability, maintainability, and system performance.

i. Frontend Layer

The frontend layer is responsible for interacting with users and collecting patient information. It acts as the interface between the user and the system.

Key Components:

- User-friendly interface
- Input forms for entering patient details
- Display section for showing prediction results

Technologies Used:

- HTML for structure
- CSS for styling
- Bootstrap for responsive design

Functions:

- Collect patient data such as age, symptoms, and department
- Send input data to the backend server
- Display predicted urgency level and waiting time

The frontend is designed to be simple, intuitive, and accessible for both patients and healthcare staff.

ii. Backend Layer

The backend layer is responsible for processing data, running machine learning models, and managing the database.

Key Components:

- Flask server for handling requests
- Machine learning models for prediction

- SQLite database for storing patient and appointment data

Functions:

- Receives input data from the frontend
- Performs data preprocessing
- Executes prediction algorithms
- Stores and retrieves data from the database
- Sends output back to the frontend

This layer forms the core of the system and ensures accurate and efficient processing.

6.2 Modules

The system is divided into multiple functional modules, each responsible for specific tasks. This modular approach improves system organization and maintainability.

i. Patient Module

- Collects patient details such as age, symptoms, and department
- Validates input data before processing
- Stores patient information in the database

ii. Prediction Module

- Uses machine learning algorithms to analyze patient data
- Predicts urgency level (low, medium, high)
- Estimates waiting time based on system load

iii. Appointment Module

- Automatically schedules appointments based on prediction results
- Allocates appropriate time slots
- Ensures efficient patient flow

iv. Notification Module

- Sends email notifications to patients
- Provides appointment confirmation details
- Updates patients regarding scheduling changes

v. Admin Module

- Allows administrators to view and manage appointments
- Provides access to patient records
- Helps in monitoring system performance

6.3 Algorithm Workflow

The PredictCare system follows a systematic workflow to process patient data and generate outputs efficiently.

Step-by-Step Workflow:

Step-1: Input Patient Details

The user enters patient information such as age, symptoms, and department through the interface.

Step-2: Data Preprocessing

Input data is cleaned and transformed into a suitable format (encoding and normalization).

Step-3: Urgency Prediction

The system uses classification algorithms to determine the urgency level of the patient.

Step-4: Waiting Time Estimation

Regression models estimate the expected waiting time.

Step-5: Doctor Recommendation

Based on the department and urgency, the system suggests suitable doctors.

Step-6: Appointment Booking

The system automatically schedules an appointment based on availability.

Step-7: Email Notification

A confirmation message is sent to the patient via email.

Workflow Representation

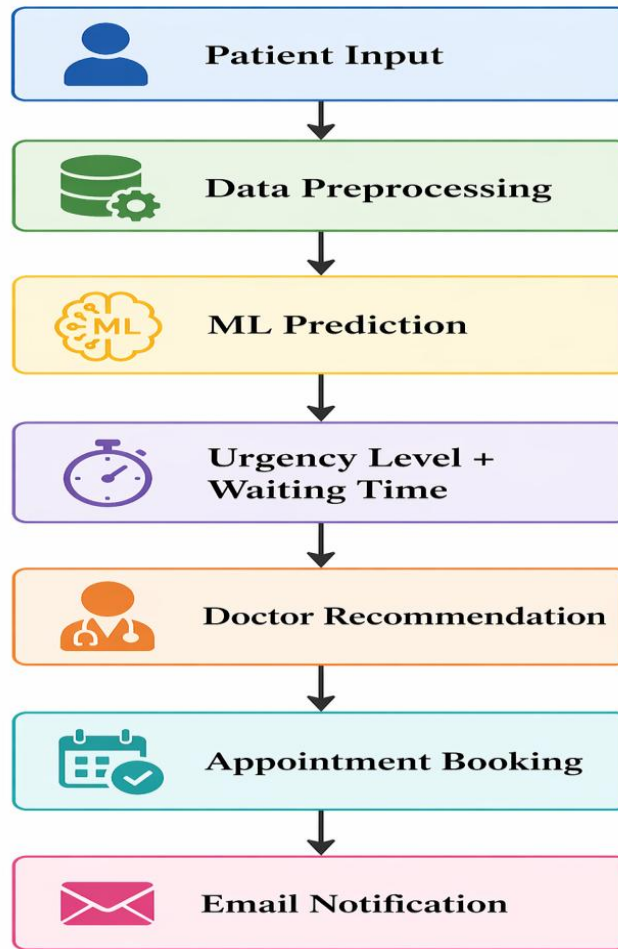


Fig.6.2:Workflow Representation

Chapter 7

RESULTS AND DISCUSSION

7.1 Overview

The PredictCare system was tested using a structured dataset representing various patient conditions. The dataset included parameters such as age, symptoms, and department, which were used as inputs for the machine learning models.

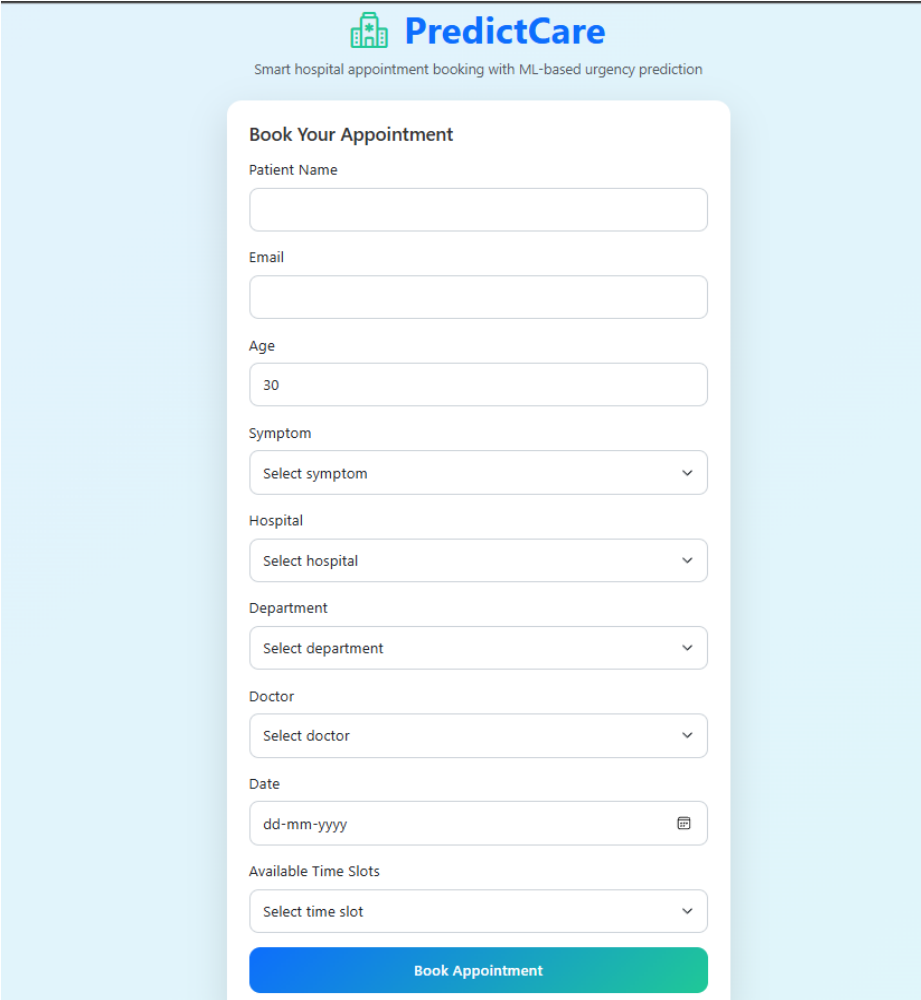
The objective of testing was to evaluate:

- The accuracy of urgency prediction
- The effectiveness of waiting time estimation
- The efficiency of appointment scheduling

During testing, multiple scenarios were simulated to ensure that the system performs reliably under different conditions. The system successfully processed user inputs, generated predictions, and provided appointment scheduling outputs in real time.

The results indicate that the system is capable of handling patient data efficiently while maintaining accuracy and consistency.

7.2: Results



The screenshot displays the 'PredictCare' web interface for booking appointments. At the top, the logo features a green building icon next to the text 'PredictCare'. Below the logo, a tagline reads 'Smart hospital appointment booking with ML-based urgency prediction'. The main section is titled 'Book Your Appointment' and contains several input fields: 'Patient Name' (text box), 'Email' (text box), 'Age' (text box with '30' entered), 'Symptom' (dropdown menu with 'Select symptom'), 'Hospital' (dropdown menu with 'Select hospital'), 'Department' (dropdown menu with 'Select department'), 'Doctor' (dropdown menu with 'Select doctor'), 'Date' (text box with 'dd-mm-yyyy' and a calendar icon), and 'Available Time Slots' (dropdown menu with 'Select time slot'). A prominent blue button at the bottom is labeled 'Book Appointment'.

Fig.7.1: Login page

This Fig.7.1 shows a smart hospital appointment booking system where users can enter personal details, select symptoms, and choose hospital, department, and doctor. It uses machine learning to predict urgency and helps schedule appointments efficiently with available time slots

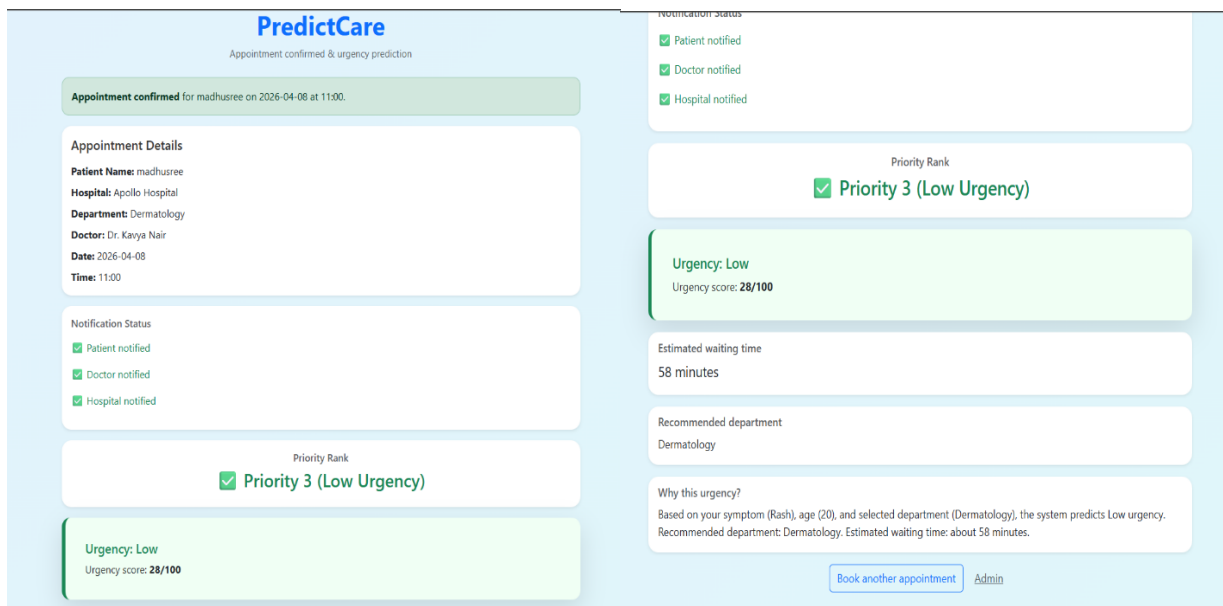


Fig.7.2: Appointment confirmation&urgency predictCare

This Fig.7.2 displays the confirmation of a booked appointment along with patient details, doctor information, and notification status. It also shows the predicted urgency level, priority rank, waiting time, and recommended department based on the patient's symptoms

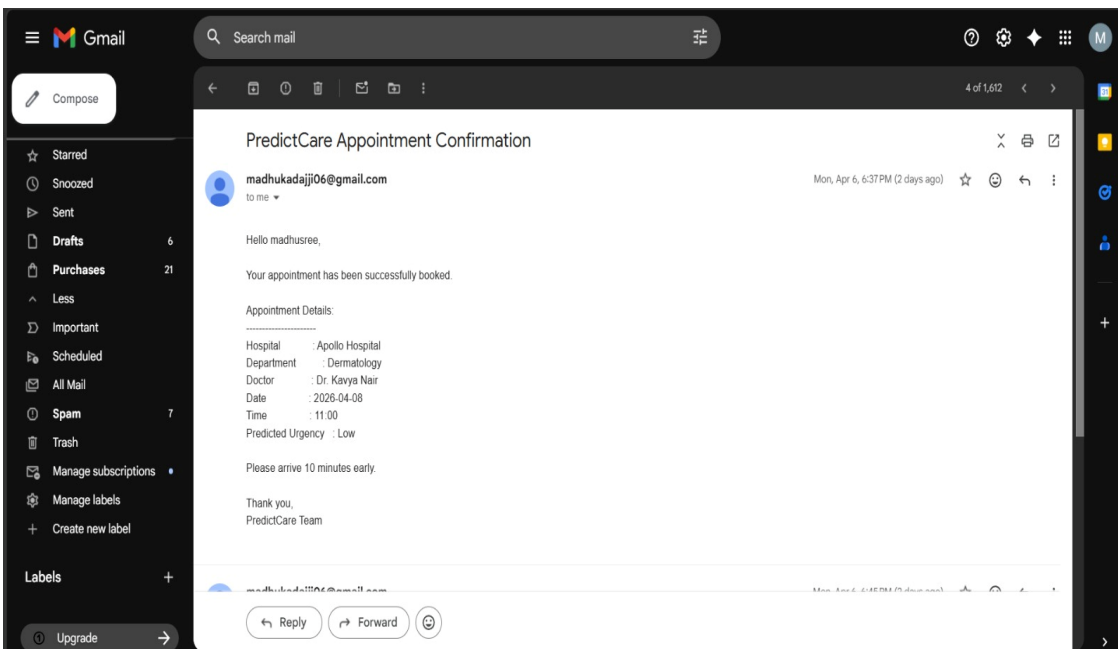


Fig.7.3 : PredictCare Appointment confirmation

This Fig.7.3 shows a Gmail confirmation email for a successfully booked PredictCare appointment with complete details like hospital, doctor, date, and time. It also includes the predicted urgency level and instructions for the patient to arrive early

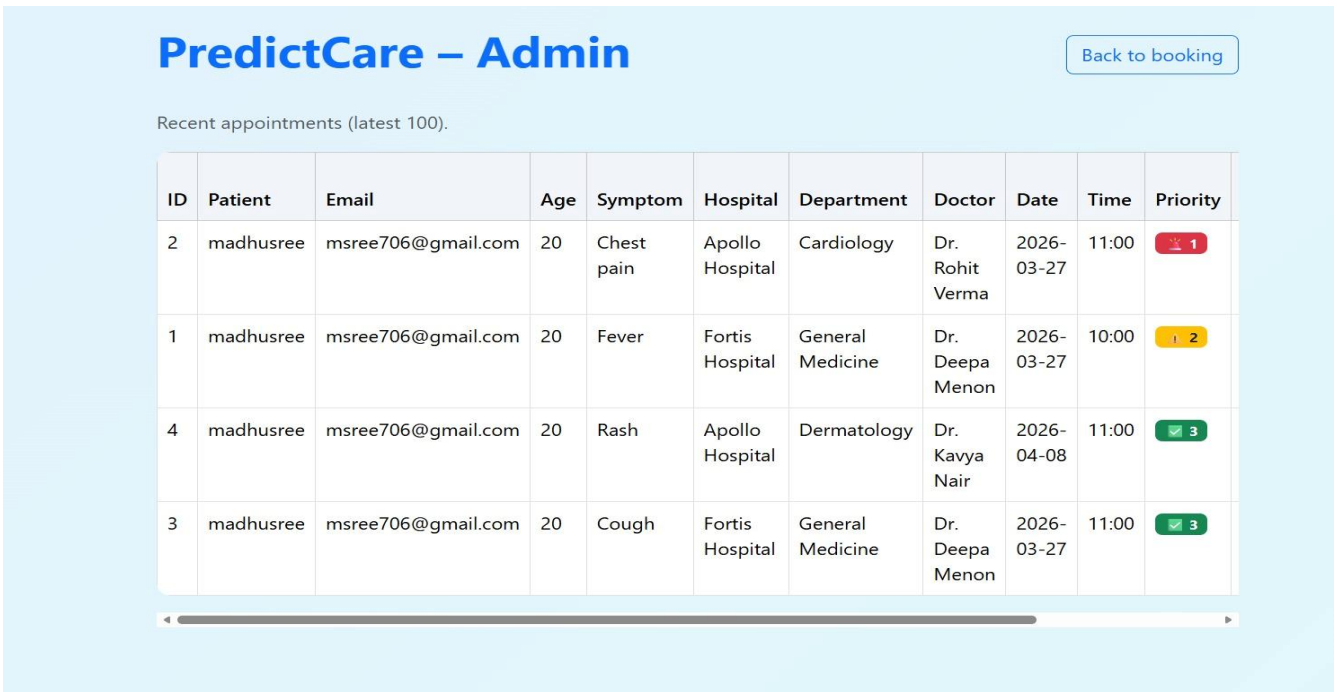


Fig.7.4: PredictCare-Admin Dashboard

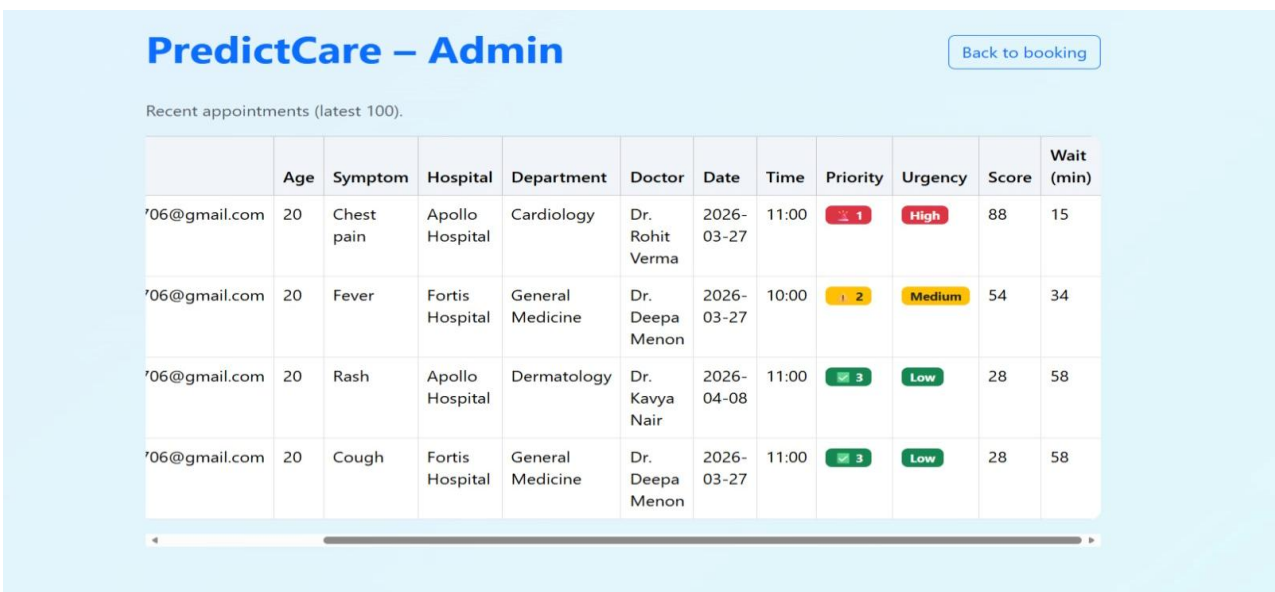


Fig.7.5 :PredictCare-Admin Dashboard

This Fig.7.5 shows the admin dashboard displaying recent patient appointments with details like symptoms, hospital, doctor, date, and time. It also highlights priority levels to help administrators quickly identify and manage urgent cases

7.3 Discussion And Analysis

The results demonstrate that integrating machine learning into healthcare systems can significantly enhance efficiency and decision-making.

Performance of Machine Learning Model

The Random Forest algorithm performed effectively due to:

- Its ability to handle both classification and regression tasks
- Robustness against overfitting
- High accuracy with structured datasets

System Scalability

The system is scalable and can be extended to:

- Handle large volumes of patient data
- Integrate with real-time hospital databases
- Support multiple users simultaneously

Real-World Applicability

PredictCare is suitable for real-world deployment with minimal modifications:

- Can be integrated into hospital management systems
- Supports automation of routine administrative tasks
- Enhances patient satisfaction through reduced waiting time

Limitations

Although the system performs well, some limitations exist:

- Depends on dataset quality and accuracy
- Requires proper training data for better predictions
- Real-time hospital integration is not yet implemented

Chapter 8

CONCLUSION

8.1 Conclusion

PredictCare successfully demonstrates how machine learning can be integrated into healthcare systems to improve patient appointment management and prioritization. By analyzing patient symptoms and related information, the system accurately predicts the urgency level of each case and estimates the expected waiting time. This enables hospitals and clinics to schedule appointments more effectively, ensuring that critical patients receive timely medical attention. The automated approach reduces dependency on manual decision-making and helps healthcare staff manage patient flow in a more organized and efficient manner.

The project also highlights the growing importance of data-driven technologies in modern healthcare. By combining predictive analytics with an intuitive user interface, PredictCare enhances operational efficiency, reduces patient waiting times, and improves overall patient satisfaction. The system demonstrates how intelligent healthcare solutions can transform traditional appointment processes into smarter, faster, and more reliable services. Overall, PredictCare serves as a practical and scalable model for future healthcare applications that leverage artificial intelligence to support better clinical and administrative decision-making.

8.2 Future Enhancements

The system can be further enhanced by integrating it with real hospital management systems and databases to enable real-time data access and improve the accuracy of predictions and scheduling processes. Developing a dedicated Android/iOS mobile application can provide users with better accessibility and convenience for booking and managing appointments. The integration of an AI-powered chatbot can assist patients by answering queries, guiding them through the booking process, and providing instant support. Real-time data processing can also be implemented to handle live patient information and generate instant predictions or updates efficiently. Furthermore, advanced deep learning models and AI techniques can be incorporated to improve system performance, prediction accuracy, and overall user experience.

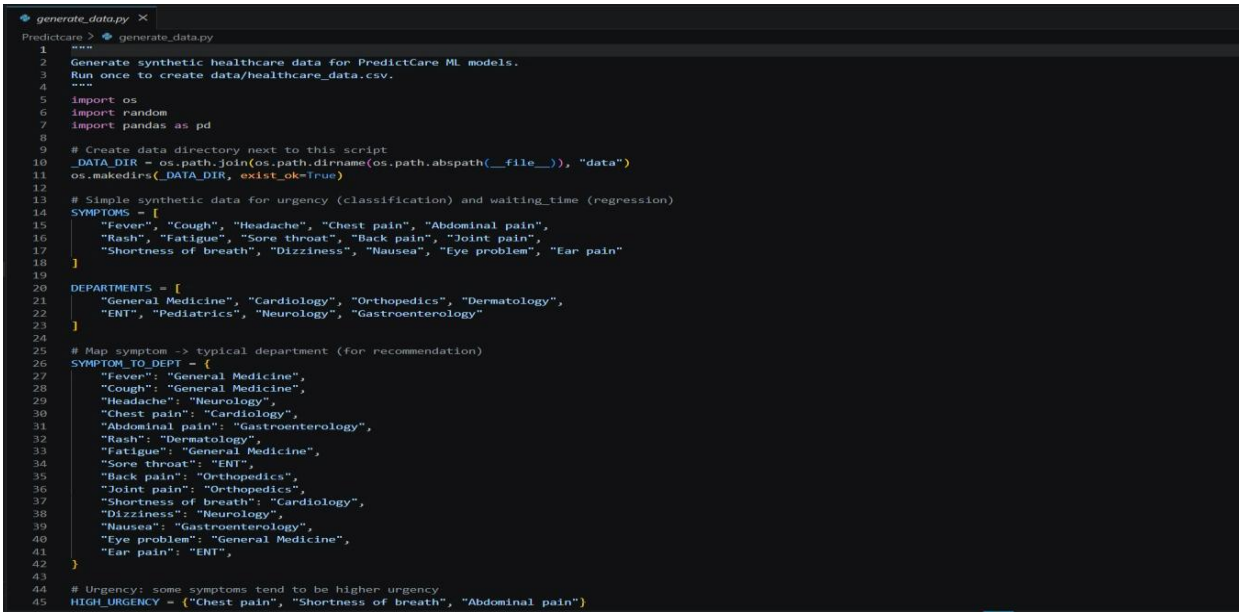
REFERENCES

- [1] Valenzuela-Núñez, C., Latorre-Núñez, G., & Troncoso Espinosa, F. Smart Medical Appointment Scheduling: Optimization, Machine Learning and Overbooking to Enhance Resource Utilization. IEEE Access, 2023. Smart_Medical_Appointment_Scheduling_Optimization_.pdf
- [2] Toffaha, K. M., Simsekler, M. C. E., Alshehhi, A., & Omar, M. A. Predicting Hospital No-Shows: Interpretable Machine Learning Models Approach. IEEE Access, 2023.
- [3] Predicting_Hospital_No-Shows_Interpretable_Machine.pdf Research on patient no-show prediction using machine learning techniques (Decision Trees, SVM, Neural Networks) for healthcare optimization.
- [4] Smart_Medical_Appointment_Scheduling_Optimization_.pdf Studies on overbooking strategies in healthcare systems to improve resource utilization and time. Smart_Medical_Appointment_Scheduling_Optimization_.pdf
- [5] Analysis of Brazilian Medical Appointment No-Show dataset for predicting attendance behavior in hospitals Predicting_Hospital_No-Shows_Interpretable_Machine.pdf
- [6] Research on interpretable machine learning models (Rule-based, Tree-based, and Algebraic models) in healthcare analytics. Predicting_Hospital_No-Shows_Interpretable_Machine.pdf
- [7] Studies highlighting factors affecting patient no-shows such as lead time, patient history, age, and distance from hospital. Predicting_Hospital_No-Shows_Interpretable_Machine.pdf

Our Project presents: <https://github.com/madhuuu06/Predictcare>

Appendix A

SCREENSHOTS



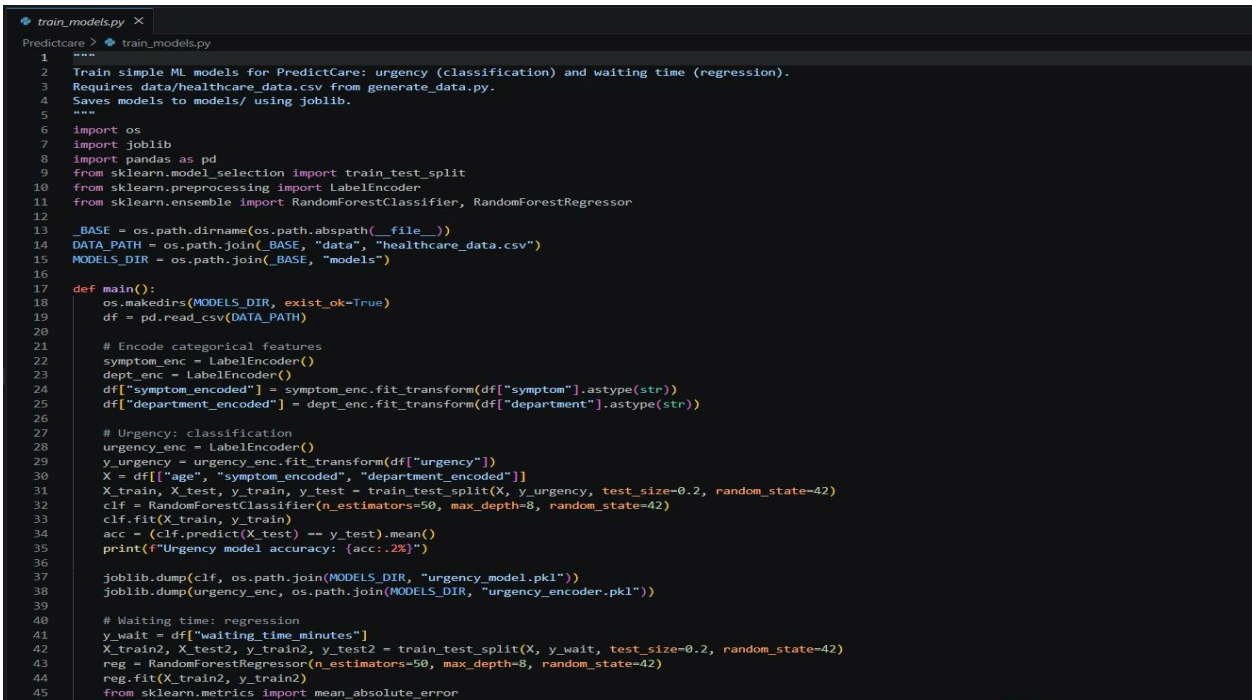
```

1  generate_data.py
2  PredictCare > generate_data.py
3  ---
4  Generate synthetic healthcare data for PredictCare ML models.
5  Run once to create data/healthcare_data.csv.
6  ---
7  import os
8  import random
9  import pandas as pd
10
11 # Create data directory next to this script
12 _DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "data")
13 os.makedirs(_DATA_DIR, exist_ok=True)
14
15 # Simple synthetic data for urgency (classification) and waiting_time (regression)
16 SYMPTOMS = [
17     "Fever", "Cough", "Headache", "Chest pain", "Abdominal pain",
18     "Rash", "Fatigue", "Sore throat", "Back pain", "Joint pain",
19     "Shortness of breath", "Dizziness", "Nausea", "Eye problem", "Ear pain"
20 ]
21
22 DEPARTMENTS = [
23     "General Medicine", "Cardiology", "Orthopedics", "Dermatology",
24     "ENT", "Pediatrics", "Neurology", "Gastroenterology"
25 ]
26
27 # Map symptom -> typical department (for recommendation)
28 SYMPTOM_TO_DEPT = {
29     "Fever": "General Medicine",
30     "Cough": "General Medicine",
31     "Headache": "Neurology",
32     "Chest pain": "Cardiology",
33     "Abdominal pain": "Gastroenterology",
34     "Rash": "Dermatology",
35     "Fatigue": "General Medicine",
36     "Sore throat": "ENT",
37     "Back pain": "Orthopedics",
38     "Joint pain": "Orthopedics",
39     "Shortness of breath": "Cardiology",
40     "Dizziness": "Neurology",
41     "Nausea": "Gastroenterology",
42     "Eye problem": "General Medicine",
43     "Ear pain": "ENT",
44 }
45
46 # Urgency: some symptoms tend to be higher urgency
47 HIGH_URGENCY = {"Chest pain", "Shortness of breath", "Abdominal pain"}

```

Fig.A.1 : Synthetic Healthcare Data Generation Module

Scn.1: The **generate_data.py** file is used to create synthetic healthcare data for training purposes. It defines lists of symptoms and hospital departments and maps symptoms to appropriate departments. It also marks certain symptoms as high urgency and generates random data accordingly, which is then saved as a CSV file. Another important function in the system is **get_available_slots()**, which retrieves available appointment time slots. It takes inputs such as hospital, doctor, and appointment date, fetches already booked slots from the database, and returns only the remaining available slots in JSON format. Overall, the system follows a pipeline where data is generated, models are trained and saved, and then used in a web application to predict patient urgency, estimate waiting time, and efficiently manage appointment bookings.



```

1  """
2  Train simple ML models for PredictCare: urgency (classification) and waiting time (regression).
3  Requires data/healthcare_data.csv from generate_data.py.
4  Saves models to models/ using joblib.
5  """
6  import os
7  import joblib
8  import pandas as pd
9  from sklearn.model_selection import train_test_split
10 from sklearn.preprocessing import LabelEncoder
11 from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
12
13 _BASE = os.path.dirname(os.path.abspath(__file__))
14 DATA_PATH = os.path.join(_BASE, "data", "healthcare_data.csv")
15 MODELS_DIR = os.path.join(_BASE, "models")
16
17 def main():
18     os.makedirs(MODELS_DIR, exist_ok=True)
19     df = pd.read_csv(DATA_PATH)
20
21     # Encode categorical features
22     symptom_enc = LabelEncoder()
23     dept_enc = LabelEncoder()
24     df["symptom_encoded"] = symptom_enc.fit_transform(df["symptom"].astype(str))
25     df["department_encoded"] = dept_enc.fit_transform(df["department"].astype(str))
26
27     # Urgency: classification
28     urgency_enc = LabelEncoder()
29     y_urgency = urgency_enc.fit_transform(df["urgency"])
30     X = df[["age", "symptom_encoded", "department_encoded"]]
31     X_train, X_test, y_train, y_test = train_test_split(X, y_urgency, test_size=0.2, random_state=42)
32     clf = RandomForestClassifier(n_estimators=50, max_depth=8, random_state=42)
33     clf.fit(X_train, y_train)
34     acc = (clf.predict(X_test) == y_test).mean()
35     print(f"Urgency model accuracy: {acc:.2%}")
36
37     joblib.dump(clf, os.path.join(MODELS_DIR, "urgency_model.pkl"))
38     joblib.dump(urgency_enc, os.path.join(MODELS_DIR, "urgency_encoder.pkl"))
39
40     # Waiting time: regression
41     y_wait = df["waiting_time_minutes"]
42     X_train2, X_test2, y_train2, y_test2 = train_test_split(X, y_wait, test_size=0.2, random_state=42)
43     reg = RandomForestRegressor(n_estimators=50, max_depth=8, random_state=42)
44     reg.fit(X_train2, y_train2)
45     from sklearn.metrics import mean_absolute_error

```

Fig.A.2 : Machine Learning Model Training Module

Scn.2: The project consists of multiple Python modules that work together to build an intelligent healthcare appointment system. The train_models.py file is responsible for training machine learning models using healthcare data stored in a CSV file. It preprocesses the data by converting categorical values like symptoms and departments into numerical form using label encoding. It then splits the dataset into training and testing sets and uses a Random Forest Classifier to predict patient urgency levels such as Low, Medium, or High. The model's accuracy is evaluated, and both the trained model and encoders are saved for later use. Additionally, a Random Forest Regressor is trained to estimate patient waiting time based on the same features, and this model is also saved.

```

app.py
Predictcare > app.py
270
271 def predict_urgency_and_wait(age, symptom, department):
272     symptom_enc = encoders["symptom"]
273     dept_enc = encoders["department"]
274
275     try:
276         s_code = symptom_enc.transform([symptom])[0]
277     except Exception:
278         s_code = 0
279
280     try:
281         d_code = dept_enc.transform([department])[0]
282     except Exception:
283         d_code = 0
284
285     X = [[age, s_code, d_code]]
286
287     urgency_idx = urgency_model.predict(X)[0]
288     urgency_label = urgency_encoder.inverse_transform([urgency_idx])[0]
289
290     wait_min = max(5, min(120, int(waiting_time_model.predict(X)[0])))
291
292     recommended = SYMPTOM_RECOMMENDATION.get(symptom, department)
293
294     score_map = {"Low": 25, "Medium": 55, "High": 88}
295     base_score = score_map.get(urgency_label, 50)
296     urgency_score = max(0, min(100, base_score + (wait_min % 10) - 5))
297
298     explanation = (
299         f"Based on your symptom ({symptom}), age ({age}), and selected department ({department}), "
300         f"the system predicts {urgency_label} urgency. "
301         f"Recommended department: {recommended}. "
302         f"Estimated waiting time: about {wait_min} minutes."
303     )
304
305     return urgency_label, urgency_score, wait_min, recommended, explanation
306

```

Fig.A.3: Urgency Prediction code

Scn.3: The **app.py** file handles the main application logic, including appointment booking and prediction. When a user tries to book an appointment, the system first checks whether the selected doctor is already booked for the chosen date and time. If the slot is unavailable, it redirects the user with an error message; otherwise, it proceeds to predict urgency and waiting time using the trained models. The system then assigns a priority rank based on urgency and stores all appointment details, including predictions, into the database. The **predict_urgency_and_wait()** function plays a key role by taking user inputs such as age, symptom, and department, encoding them into numerical values, and feeding them into the trained models. It returns the predicted urgency label, urgency score, estimated waiting time (bounded between 5 and 120 minutes), a recommended department, and a user-friendly explanation of the results.

```
app.py ×
Predictcare > app.py
315 @app.route("/get_available_slots")
316 def get_available_slots():
317     hospital = request.args.get("hospital", "").strip()
318     doctor = request.args.get("doctor", "").strip()
319     appointment_date = request.args.get("appointment_date", "").strip()
320
321     if not hospital or not doctor or not appointment_date:
322         return jsonify([])
323
324     conn = get_db()
325     booked_rows = conn.execute("""
326         SELECT appointment_time FROM appointments
327         WHERE hospital = ? AND doctor = ? AND appointment_date = ?
328     """, (hospital, doctor, appointment_date)).fetchall()
329     conn.close()
330
331     booked_slots = [row["appointment_time"] for row in booked_rows]
332     available_slots = [slot for slot in ALL_TIME_SLOTS if slot not in booked_slots]
333
334     return jsonify(available_slots)
335
336
```

Fig.A.4 : Available Slots code

Scn.4: The `get_available_slots()` function is used to display the available appointment time slots for a selected doctor on a particular date. It takes the hospital name, doctor name, and appointment date as input from the user. If any of these inputs are missing, it returns an empty list. Otherwise, it connects to the database and retrieves all the already booked time slots for that doctor. It then compares these booked slots with the predefined list of all time slots and returns only the slots that are not yet booked. This helps in preventing double booking and ensures proper appointment scheduling.

```

app.py
Predictcare > app.py
338 def book():
356     existing_slot = conn.execute("""
357         SELECT * FROM appointments
358         WHERE hospital = ?
359         AND doctor = ?
360         AND appointment_date = ?
361         AND appointment_time = ?
362     """, (hospital, doctor, appt_date, appt_time)).fetchone()
363
364     if existing_slot:
365         conn.close()
366         return redirect(url_for(
367             "index",
368             slot_error="This doctor is already booked for the selected date and time. Please choose another doctor or time slot."
369         ))
370
371     urgency_label, urgency_score, wait_min, recommended, explanation = predict_urgency_and_wait(
372         age, symptom, department
373     )
374
375     priority_rank = get_priority_rank(urgency_label)
376
377     cur = conn.execute(
378         """INSERT INTO appointments (
379             patient_name, email, age, symptom, hospital, department, doctor, appointment_date, appointment_time,
380             urgency, priority_rank, urgency_score, waiting_time_minutes, recommended_department, explanation
381         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
382         (
383             name, email, age, symptom, hospital, department, doctor, appt_date, appt_time,
384             urgency_label, priority_rank, urgency_score, wait_min, recommended, explanation
385         ),
386     )
387     appt_id = cur.lastrowid

```

Fig.A.5: Booking Validation code

This screenshot shows the appointment booking logic in app.py. The system first checks if the selected doctor is already booked for the chosen date and time. If the slot is unavailable, it returns an error message. Otherwise, it proceeds to predict urgency, assign a priority rank, and store the appointment details in the database.

Appendix B

SOURCE CODE

PredictCare: Patient Urgency and Waiting Time Prediction System

```
from flask import Flask, request, render_template_string
import pandas as pd
import random
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

# Initialize Flask application
app = Flask(__name__)

# -----
# STEP 1: GENERATE SYNTHETIC DATA
# -----

symptoms = ['fever', 'cold', 'chest pain', 'fracture', 'headache']
departments_map = {
    'fever': 'General',
    'cold': 'General',
    'chest pain': 'Cardiology',
    'fracture': 'Orthopedics',
    'headache': 'Neurology'
}

data = []

for _ in range(200):
    symptom = random.choice(symptoms)
    age = random.randint(1, 80)
```

```
# Assign urgency and waiting time based on symptom
if symptom in ['chest pain', 'fracture']:
    urgency = 'High'
    wait = random.randint(5, 20)
elif symptom == 'fever':
    urgency = 'Medium'
    wait = random.randint(20, 40)
else:
    urgency = 'Low'
    wait = random.randint(40, 60)

department = departments_map[symptom]

data.append([age, symptom, department, urgency, wait])

# Create DataFrame
df = pd.DataFrame(
    data,
    columns=['age', 'symptom', 'department', 'urgency', 'waiting_time']
)

# -----
# STEP 2: PREPROCESSING
# -----

le_symptom = LabelEncoder()
le_dept = LabelEncoder()
le_urgency = LabelEncoder()

df['symptom'] = le_symptom.fit_transform(df['symptom'])
df['department'] = le_dept.fit_transform(df['department'])
df['urgency'] = le_urgency.fit_transform(df['urgency'])
```

```
# Features and Targets
X = df[['age', 'symptom', 'department']]
y_urgency = df['urgency']
y_waiting_time = df['waiting_time']

# -----
# STEP 3: TRAIN MODELS
# -----

# Random Forest Classifier for urgency prediction
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X, y_urgency)

# Random Forest Regressor for waiting time prediction
reg = RandomForestRegressor(n_estimators=100, random_state=42)
reg.fit(X, y_waiting_time)

# -----
# STEP 4: HTML TEMPLATE
# -----

html_page = """
<!DOCTYPE html>
<html>
<head>
    <title>PredictCare System</title>
</head>
<body style="font-family: Arial; text-align: center; margin-top: 50px;">
    <h1>PredictCare System</h1>

    <form method="POST">
        <input type="number" name="age" placeholder="Enter Age" required><br><br>
```

```

<select name="symptom">
  <option value="fever">Fever</option>
  <option value="cold">Cold</option>
  <option value="chest pain">Chest Pain</option>
  <option value="fracture">Fracture</option>
  <option value="headache">Headache</option>
</select><br><br>

<select name="department">
  <option value="General">General</option>
  <option value="Cardiology">Cardiology</option>
  <option value="Orthopedics">Orthopedics</option>
  <option value="Neurology">Neurology</option>
</select><br><br>

<button type="submit">Predict</button>
</form>

{% if result %}
  <h2>Urgency Level: {{ result[0] }}</h2>
  <h3>Waiting Time: {{ result[1] }} minutes</h3>
{% endif %}
</body>
</html>
"""

# -----
# STEP 5: ROUTES
# -----

@app.route('/', methods=['GET', 'POST'])
def home():
    result = None

```

```
if request.method == 'POST':
    age = int(request.form['age'])
    symptom = request.form['symptom']
    department = request.form['department']

    # Encode inputs
    symptom_encoded = le_symptom.transform([symptom])[0]
    dept_encoded = le_dept.transform([department])[0]

    # Prepare input data
    input_data = [[age, symptom_encoded, dept_encoded]]

    # Predict urgency and waiting time
    urgency_pred = clf.predict(input_data)[0]
    wait_pred = int(reg.predict(input_data)[0])

    # Decode urgency prediction
    urgency = le_urgency.inverse_transform([urgency_pred])[0]

    result = (urgency, wait_pred)

    return render_template_string(html_page, result=result)

# -----
# STEP 6: RUN APPLICATION
# -----

if __name__ == '__main__':
    app.run(debug=True)
```